

Doc Intelligence: Design Notes

Doc Intelligence answers questions about PDFs with citations that point to the exact page. It is a retrieval-augmented generation (RAG) system: the model never answers from memory, only from excerpts retrieved out of the documents a user has uploaded.

Ingestion. An upload is stored and acknowledged immediately; the heavy work runs in the background. Text is extracted page by page, split into chunks of roughly 400 tokens that overlap by about 60, embedded with Voyage (voyage-3, 1024 dimensions) and written to Postgres in a single transaction. A document is never marked ready while only part of it is embedded, because answering from partial evidence is worse than failing.

Deduplication. Every upload is hashed with SHA-256. Uploading the same bytes twice, even under a different filename, reuses the existing document instead of paying to embed it again.

Scanned PDFs contain images rather than text. They fail ingestion with a clear message saying OCR is needed, rather than silently producing an empty index.

Retrieval and Answers

Vectors live in Postgres through the pgvector extension, indexed with IVFFlat over cosine distance. Keeping vectors next to the relational data means one database, one backup and one transaction boundary instead of a separate vector store to keep in sync.

A question is embedded with the query input type, and the eight most similar chunks are retrieved. Voyage encodes documents and queries with different prefixes; using the document type for both measurably lowers recall.

The answer streams back over server-sent events, token by token. The model is instructed to cite excerpts as [1], [2] and so on, and each marker resolves to a filename and a page range the reader can verify.

Identical questions over the same documents are cached for six hours. The cache key includes the user id, because the same question asked by two users has two different correct answers.

Security and Limits

Tenant isolation. Every document and chunk query filters on the authenticated user id. That WHERE clause is the tenant boundary, which is why the chunks table carries its own user_id column and index rather than joining through documents.

Authentication uses bcrypt-hashed passwords and signed JWTs. A login for an unknown email still performs a bcrypt comparison, so a wrong email and a wrong password take the same time to answer.

Rate limits. Asking questions is limited to 60 per user per hour, because every answer spends real model quota. Sign-in attempts are limited per IP address to slow down credential stuffing. When Redis is unavailable the limiter fails open rather than taking the API down.

Demo accounts are created with one click, start with a copy of this document, and are deleted automatically after two days.